

The Query Language Vizla

Alfs T. Berztiss

Abstract—We define the query language Vizla, in which answers to queries are built up by pointing to representations of sets and functions in a conceptual model of the data base of an application, and to iconic identifiers of computational operators or control constructs. At its present stage of development the primary use of Vizla is in the validation of conceptual models of information systems, but it is to be developed into a user interface to SF, a prototyping language for information and control systems. Moreover, it can be regarded as a visual programming language in its own right. As such it is based on abstract data types.

Index Terms—Conceptual model, data type, information system, prototyping environment, query formulation, visual language.

I. INTRODUCTION

THE PRIMARY problem confronting developers of computerized systems is the proper balancing of rigorous and unambiguous definition with user convenience. This problem extends from the specification of the system to its use. Our purpose here is to look at how this problem manifests itself in the formulation of queries to data base systems. We shall examine the problem, look briefly at past efforts at a solution, and give our own solution, which is the visual language Vizla. Although Vizla is quite general, the motivation for its development came from prototyping of information systems. This we see as its main application area.

We shall assume that the data base of an information system consists of a collection of finite sets and functions. For example, an employee data base may consist of a set (of representations) of employees, and of finite functions from this set to auxiliary sets, such as (representations of) telephones, addresses, dates of birth, etc. Access to this data base is by look-up or by the application of some computational rule to the stored data. For example, a date of birth can be looked up directly, i.e., it is a primary datum, but an age is a secondary datum determined by a computation involving the date of birth and the current date. Look-up can be initiated by supplying a function name and an appropriate argument; a computational rule has to be specified explicitly or implicitly, and such specification is our main concern here.

It has been suggested that two definitions of a system are needed. One describes it in terms that a client or user

understands; the other relates to the needs of the software developer. Dieter Rombach [1] calls them C- (for client) and D- (for designer) specifications, respectively. An immediate problem is how to maintain consistency between the two specifications. Consistency can be guaranteed only by a formal proof or by the generation of the D-specification from the C-specification by correctness-preserving transformations. In either case this requires that the C-specification be expressed in a formal language, i.e., a language with formally defined syntax and semantics. But then one might try to make the C-specification executable, which means turning it into a prototype.

Formal prototyping languages with a fair degree of client orientation have been developed (e.g., JSD [2], ERAE [3], INFOLOG [4], USE [5], SF [6], which are all briefly described in [7]). The prototype of an information system differs from its final version in that it may be inefficient, and may lack ancillary features such as prompt and help systems. However, the structure of the data base of the final system must be faithfully reflected in the prototype. The whole point of prototyping is that it allows the client to determine whether the data base of the system is adequate for answering all queries that are expected to be put to the system when it becomes operational.

A facility for the formulation of queries must therefore be an integral component of an information system from its earliest development stages, and the query language has to satisfy formality requirements to no less extent than the prototyping language. Consequently, natural language is inappropriate, but, on the other hand, the formulation of queries as expressions in logic does not satisfy the requirement of client convenience.

To take a simple example, consider a conference. Authors submit papers, but some authors may be prevented from attending the conference, and the presenter of a paper may then be different from an author. Consider now the query "Find all presenters of papers who are not authors." This query is ambiguous in that the term author may relate to four different classes of written material: a) the particular paper whose presentation is under discussion, b) any accepted paper, c) any submitted paper, or d) anything at all intended for publication. As a further illustration of the interpretation difficulties, suppose A presents paper X of which A is not the author. Let S be the set that is the answer to the query under interpretation a). We would put A in S. But now suppose that A is an author of paper Y, and also presents this paper. Should A still be a member of S? This is a variant of interpretation a) that we shall not pursue further.

The ambiguity here has little to do with synonyms or other sources of linguistic ambiguity—it arises out of application-

Manuscript received July 20, 1990; revised October 1, 1991 and April 24, 1992. This work was partially supported by STU (Swedish National Board of Technical Development) and AITRC (Applied Information Technologies Research Center).

The author is with the Department of Computer Science, University of Pittsburgh, Pittsburgh, PA 15260, and SYSLAB, University of Stockholm, Sweden.

IEEE Log Number 9211307.

dependent interpretation of components of the data base. The logical specifications of the answers to the query distinguish clearly between the four interpretations, but they are not exactly user-friendly:

- (a) $\{y|\exists x : x \in ACC : y = Pres(x) \wedge y \notin Auth(x)\},$
- (b) $\{y|\exists x : x \in ACC : y = Pres(x)\}$
 $- \{y|\exists x : x \in ACC : y = Auth(x)\},$
- (c) $\{y|\exists x : x \in ACC : y = Pres(x)\}$
 $- \{y|\exists x : x \in PAP : y = Auth(x)\},$
- (d) $\{y|\exists x : x \in ACC : y = Pres(x)\} - AUT.$

Here *PAP* is the set of papers submitted to the conference, *ACC* the subset of accepted papers, and *AUT* the general set of authors. *Pres* is a total function from *ACC* to attendees of the conference, and *Auth* is a total set-valued function from *PAP* to the power-set of *SUB*, where the latter is the set of submitters of papers to the conference, a subset of *AUT*.

We shall develop a visual representation of the data base in which the distinction between the four interpretations is reasonably well defined. The four queries are then easily formulated by pointing to parts of this representation.

Our choice of functions rather than relations in the definition of data bases was made for the sake of uniformity. The concept of a function is basic to programming. In some contexts, such as functional programming or data abstraction, it is the dominant concept. Specifically our choice ensures that at a sufficiently high level of abstraction there is no difference between the response to a query, particularly when the response requires the use of a computational rule, and the evaluation of a function in conventional programming. In both instances arguments are supplied and values returned. Under this interpretation a query language should be able to deal with queries of arbitrary complexity, i.e., it should have the power of a general programming language.

Section II gives a brief survey of visual query languages, and Section III introduces our languages SF and Vizla. Detailed discussion of the components of Vizla follows as Section IV. A rather complicated query is studied in Section V. Extensions are considered in Section VI, and our work summarized in Section VII. Although Vizla is not intended to be used as a general programming language, it does allow for queries of arbitrary complexity. We demonstrate its power by example: the Appendix presents a Vizla program for Dijkstra's shortest path algorithm.

II. A BRIEF HISTORY

We shall not examine the entire spectrum of query languages because this has already been done in an outstanding way by Jarke and Vassiliou [8]. Instead we will adduce evidence that most users experience difficulties with textual query languages

based on logic. This will lead us to support visual query languages.

In general, humans prefer procedural over nonprocedural approaches to problem solving. Difficulties in the use of nonprocedural approaches, of which logical formulations are a particular example, have long been recognized. For example, students presented with a simple word problem made more mistakes in expressing the problem as a nonprocedural equation than in devising an algorithm for its solution [9]. Further, users have shown greater satisfaction with relational algebra (procedural) than with relational calculus (declarative) [10], [11]. Difficulties with nonprocedural formulations affect reading as well as writing. Indeed, it has been necessary to provide English paraphrases of Gist specifications [12], and natural language paraphrasing is still needed when a query language has been given a semblance to a natural language, as is the case with SQL [13].

It seems that the most effective way to combine formality with understandability is by visual means. A visual query language based on a display of the structure of a data base holds the intuitive appeal of natural language, but, because a query is formulated by well-defined composition of components in the pictorial representation of the data base, the visual language can be as formal as logic. Numerous attempts have been made of defining visual query languages (see, e.g., [14]–[21]), but the operators for the composition of the components of the pictorial representations have been rather primitive.

Larson [22] provides a visual interface for query formulation that is more elaborate than the norm. Here the data base is modeled directly by an entity-relationship schema, and the user navigates through a visual representation of the schema. Operators for composing a query are selected from the syntax flow diagram of the query language. The example provided by Larson refers to a data base that models a university at which students take courses that bear numbers, and a course can have more than one section. In the example it is shown how one builds the query

Get student where SectionNumber = 3

and CourseNumber = 177.

The building blocks are the usual arithmetic operators, comparison operators, and the boolean operators "not," "or," and "and." It appears though that the expressive power of the query language may not live up to the expectations of a sophisticated user. The resulting queries are in effect expressions in logic, the system merely guiding the user in the correct application of syntax. However, the entity-relationship diagram is a great help to the user.

The query language of Kangassalo's group at the University of Tampere [23] is based on similar principles. Here the query is constructed with the help of a visual conceptual model defined in the Concept-D language [24], and the system translates the query into SQL. As with Larson's approach, standard arithmetic and logical operators are provided.

In our investigation we start out with an approach similar to those of Larson and the Tampere group. Our intention,

however, is to create an environment in which the potential for misunderstandings of the data base by its users is explicitly recognized, and the system is deliberately designed to take this into consideration. The design is also likely to facilitate the next steps in the research, one of which will be the construction of candidate queries by the system based on a minimal input by the user.

Let us now turn to visual programming. This term has been used in many different senses, as indicated by the range of topics covered in a special issue of *Computer on visual programming* [25]: static and dynamic program visualization, displays of tabular forms, use of state transition diagrams, and graphic design tools.

Our concern is a query language that has the computational power of a programming language. Hence the discussion here will be limited to graphic approaches to program development. In general, two approaches to program development have been taken. The earlier of the two, and perhaps still the more prevalent, has emphasized computation as a process. A more recent development puts emphasis on data. The distinction between the two approaches was analyzed by Parnas as early as 1972 [26], with an endorsement of the data-oriented approach. Under this approach, data are partitioned into types, where each type is a set and operations on elements of the set. A program consists of the operations arranged into execution patterns by means of control constructs.

Construction of programs by visual means has been confined to the process-oriented approach, and there have been some notable successes, in particular the conversion of Jackson System Development (JSD) diagrams to executable code in a variety of programming languages [27]. Prototyping of distributed systems is another area of success [28], [29]. To our knowledge, nothing comparable has been reported for the data-oriented approach.

III. AN INTRODUCTION TO SF AND VIZLA

An important use of Vizla is in the validation of conceptual models of information systems: a conceptual schema is drawn, and users check that the information base represented by this schema is adequate for answering all queries that representative users put to the system. If not, more components have to be added. This can be done in a paper-and-pencil mode, but our intention is to make Vizla the query language for SF, an executable specification language for information-control systems. We shall therefore start with an SF schema, and from it derive the diagram on which Vizla operates.

An SF specification defines a collection of interacting modules, where each module corresponds to a data type. An SF type consists of a universe of entities T , a finite set of interest S that consists of objects from T , and a collection of functions. For convenience we shall use T to denote both the type and its universe of entities. In an information system for a taxi company one type would encompass all automobiles ever built or to be built, but the corresponding set S would consist of automobiles the company either owns or has owned. The functions map from S or some subset of S to a type, a cartesian product of types, or the power set of a type or of a cartesian

product, i.e., the value of a function may be an element, a tuple, or a set of elements or tuples.

One of the problems faced by designers of languages for information systems concerns null values. For example, an employee may have no phone, although normally the employee will have a phone, or even several. In Daplex [30], the solution is to make all functions set-valued. Then, the value of the telephone number for our three cases would be the null set, a singleton set, or a set with more than one elements, respectively. This is an elegant solution to the problem of null values, but makes functional composition next to impossible. Another solution is to introduce explicit null values. Then, under composition, a function applied to a null value yields a null value, but interpretation of a function applied to a tuple containing a null value is complicated. In SF the last problem is avoided in that functions may map only from set S or some subset of S . Moreover, all SF functions are total. This is achieved in several ways. First, if functional composition is not an issue, the function can be made set-valued. Second, we have already seen that use of explicit null values does not create any problems in SF. The third solution is to use partitions of S . For example, a foreign student in the United States may not have been issued a Social Security number. The set of students is partitioned into subsets of students with and without a Social Security number, and the Social Security number function is defined on the former subset alone.

Each SF type is described by a schema. We now give a schema definition of a small SF information base relating to the example of Section I. The set identifiers are abbreviations: *PER*son, *ATT*endee, *AUT*hor, *SUB*mitter, *PAP*er, *COUN*tries. Set *PAP* is partitioned into accepted papers (*ACC*) and papers that have not been accepted (*NAC*). Function *Country* defines the country of residence of a person, *Length* defines the number of pages in a submitted paper, the set-valued function *Auth* defines the authors of each paper, and *Pres* defines the presenter of each accepted paper. An *ISA* indicates type inheritance. For example, if $X \text{ ISA } Y$, then the set of X is a subset of the set of Y , i.e., an element may be added to X only if it already belongs to Y . In a real-life example the type definitions would include a larger number of functions, as well as declarations of imported types and communication devices [31]. Note that some types, such as *Integer* (with set *INT*), are supplied by SF.

```
TYPE: Person;
SET PER;
FUNCTIONS Country: PER→Countries;
ENDTYPE;
```

```
TYPE Attendee ISA Person;
SET ATT;
ENDTYPE;
```

```
TYPE Author ISA Person;
SET AUT;
ENDTYPE;
```

```
TYPE Submitter ISA Author;
SET SUB;
ENDTYPE;
```

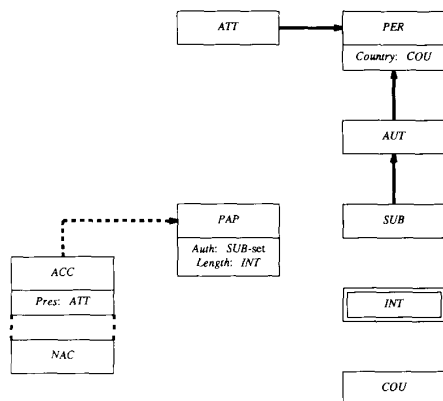


Fig. 1. Type structure for the conference example

```

TYPE Paper;
  SET PAP (SUBSETS: ACC, NAC);
  FUNCTIONS  Length: PAP→Integer;
              Auth:  PAP→Submitter-set;
              Pres:  ACC→Attendee;
ENDTYPE;

```

```

TYPE Countries;
  SET COU;
ENDTYPE;

```

Each type is the basis for a segment. In addition to the schema the segment defines events, which describe permissible changes to the sets and functions of the information base, and a control component, which allows SF events to be arranged into processes, allows the system to communicate with its environment, etc. Details can be found in [31]. If SF is used to define a prototype, then representations for the elements of the sets have to be selected. For example, a paper could be identified by a numerical code or by its title.

The purpose of SF is to deal with data base changes and the control of processes. Basic SF functions can, of course, be referenced directly, i.e., one can find the set of authors of a particular paper. However, if the user needed to know all papers longer than 10 pages that have three or more authors, of which at least one lives in Canada, a means of defining the answer to this query has to be found. This is the purpose of Vizla.

From an SF schema definition a user can easily create a type diagram such as Fig. 1 by listing the types of interest for a particular application. The diagram can be left as is. However, a user can examine the diagram and eliminate functions that are of no interest for the application. If the number of retained functions is not too large, the diagram can be redrawn with more structure shown. For Fig. 1 the result is the conceptual schema of Fig. 2, which displays also the Vizla operators. We shall refer to the representation of Fig. 2 in all that follows. Here a heavy arrow indicates ISA inheritance, heavy dashed arrows indicate the partitioning of a set into subsets, a light arrow stands for a set-valued function, and a dashed light arrow

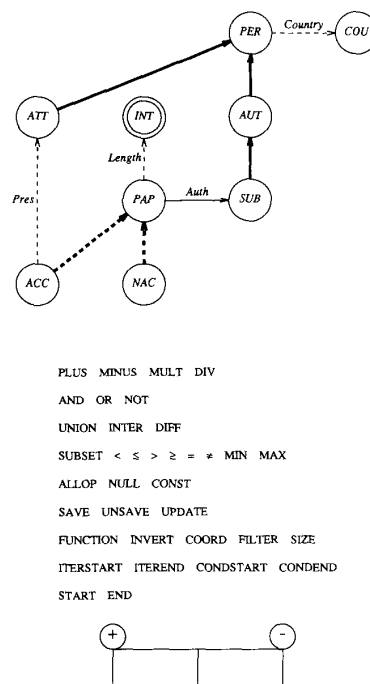


Fig. 2. Type structure for an application and Vizla operators.

for an element-valued function. The double circle on *INT* indicates that this is an infinite set.

The schema of Fig. 2, because it clearly links presenters with accepted papers, suggests our first two interpretations of the query discussed in Section I to be the more natural. Moreover, there is a general mechanical procedure for distinguishing between these two interpretations. In the first instance one examines each member x of the subset of accepted papers in turn, finds $Pres(x)$ and $Auth(x)$, and adds $Pres(x)$ to the answer if $Pres(x) \notin Auth(x)$. In the second instance the set of all authors of accepted papers is subtracted from the set of all presenters. Because of the chain leading from accepted papers to papers in general, and via function *Auth* and the ISA to authors in general, the other two interpretations can be derived from the structure as well. As a general rule, whenever an ISA hierarchy or a subset partition is encountered, the possible ambiguity of a query should be investigated.

The basic idea in Vizla is that by pointing to representations of functions, their domains and codomains, or subsets of the domains and codomains, as well as various operator symbols, new sets are assembled in a workspace. For instance, pointing to node *ACC* brings the set of accepted papers into the workspace. If *Pres* is pointed to next, each accepted paper is replaced in the workspace by its presenter. Since a person may present more than one paper, the workspace actually contains a bag rather than a set. A bag differs from a set in that elements may be duplicated—here the bag will contain as many elements as the set *ACC*. Since all presenters of papers are persons, function *Country* is applicable to our bag of presenters. Hence, by pointing next to *Country*, each

presenter is replaced by his or her country of residence. Now, suppose that after pointing to *ACC* we point out *Auth* instead of *Pres*. Then each accepted paper is replaced by its set of authors, i.e., the workspace becomes a collection of sets. Pointing to *Country* now has no effect, because *Country* maps from individual elements rather than from sets. Sets constructed in this manner can be saved as explicitly or implicitly named entities.

Let us now construct the sets that define the answers to the query of Section I under interpretations a) and b). We shall first find presenters who are not authors of any accepted paper. This requires the following sequence of actions, where $\rightarrow$ stands for "point to," and WM stands for a Workspace Monitor that displays an identifier of the current contents of the workspace:

- 1) $\rightarrow$ START [Start of retrieval session]
- 2) $\rightarrow$ ACC [WM displays ACC]
- 3) $\rightarrow$ Pres [WM displays Pres(ACC)]
- 4) $\rightarrow$ SAVE [User names this set *X*; WM unchanged]
- 5) $\rightarrow$ ACC [WM displays ACC]
- 6) $\rightarrow$ Auth [WM displays Auth(ACC)]
- 7) $\rightarrow$ ALLOP
- 8) $\rightarrow$ UNION [WM displays UNION(Auth(ACC))]
- 9) $\rightarrow$ SAVE [System names this set *9; WM unchanged]
- 10) $\rightarrow$ X [WM displays X]
- 11) $\rightarrow$ DIFF
- 12) $\rightarrow$ *9 [WM displays DIFF(*X*, *9)]

We shall refer to a sequence of actions such as the above as a querying process. Actions 2 and 3 of our querying process identify *ACC* as the set of entities for which the value of *Pres* is to be determined, and bring the set of presenters into the workspace. The SAVE of Action 4, which is a Vizla operation, preserves this set, and displays a circle representing this set next to the schema of Fig. 2. A SAVE also prompts the user: the user may give an explicit name to the set, as is done here, or the naming could be left to the system, which would use the number of the operation by which the set was created for its name (as in Action 9). The domain of the set-valued function *Auth* is the set *PAP*. However, by pointing to its subset *ACC* (Action 5), only the collection of the sets of authors of the accepted papers is built up by Action 6. Actions 7 and 8 replace the collection of sets by a single set that is the distributed union of the individual sets (UNION and ALLOP are Vizla operations; the latter will be discussed in Section IV). Actions 10–12 find the set difference of presenters and authors of accepted papers (DIFF is a Vizla operation). Note again that pointing to a node and then to a function symbol builds a bag rather than a set in the workspace. A bag can be turned into a set by, for example, application of UNION. The rules for bag to set conversion are too technical to be discussed further here. As we noted above, the Workspace Monitor does not display the actual contents of the workspace, but an identifier of the contents, e.g., the text string UNION(Auth(ACC)). A discussion of more advanced display features can be found in Section VI.

The querying process defined above is tied too closely to the structure of set definition b) of Section I, and is a rather trivial

illustration of the general observation that a new language requires the development of a new programming style. To help this development we have introduced a fair number of examples throughout. A more efficient querying process now follows:

- 1) $\rightarrow$ START
- 2) $\rightarrow$ ACC
- 3) $\rightarrow$ Auth
- 4) $\rightarrow$ ALLOP
- 5) $\rightarrow$ UNION
- 6) $\rightarrow$ SAVE
- 7) $\rightarrow$ ACC
- 8) $\rightarrow$ Pres
- 9) $\rightarrow$ DIFF
- 10) $\rightarrow$ *6

Next let us build up the set of presenters who are not authors of the particular paper, as required by interpretation a) of Section I. For this we need an iterative construct because we have to establish for each accepted paper individually that its presenter is not one of its authors. The sequence of operations:

- 1) $\rightarrow$ START
- 2) $\rightarrow$ NULL [WM displays NULL]
- 3) $\rightarrow$ SAVE [User names this *X*; WM displays NULL]
- 4) $\rightarrow$ ITERSTART
- 5) $\rightarrow$ ACC [WM displays $\in$ ACC]
- 6) $\rightarrow$ SAVE [User names this *P*]
- 7) $\rightarrow$ Auth [WM displays Auth($\in$ ACC)]
- 8) $\rightarrow$ SAVE [User names this *Y*]
- 9) $\rightarrow$ P [WM displays *P*]
- 10) $\rightarrow$ Pres [WM displays Pres(*P*)]
- 11) $\rightarrow$ DIFF [WM displays DIFF(Pres(*P*), *Y*)]
- 12) $\rightarrow$ Y [WM displays *Y*]
- 13) $\rightarrow$ UNION
- 14) $\rightarrow$ X [WM displays UNION(*Y*, *X*)]
- 15) $\rightarrow$ UPDATE
- 16) $\rightarrow$ X
- 17) $\rightarrow$ ITEREND

Here Actions 2 and 3 create the empty set *X*: Action 2 initiates the work space to a null object, and, since the workspace is to contain a set, this is the null set; Action 3 is used to give it the name *X*. Actions 4 and 17 define the scope of an iteration. Iteration is always over the elements of a set or a bag, and Action 5 designates this set or bag. Moreover, each time this action is reached in the iterative process, a new element from the set or bag is selected nondeterministically, and this element is placed as the current iteration element into the workspace. Action 6 saves the iteration element as *P*. Functions are always applied to the elements found in the workspace. Here the workspace holds a single element, which represents a paper, and *Auth* applied to it produces the set of its authors. Action 8 saves this set as *Y*. Actions 9 and 10 put the presenter of paper *P* into the workspace, but the set subtraction of Actions 11 and 12 removes it in case the presenter is also an author. Note that in Vizla an element is regarded as a singleton set whenever the context requires this. Actions 13 and 14 build up the answer set, and actions 15 and 16 overwrite set *X* with the current contents of the workspace.

The set or bag of a Vizla iteration has to be finite. Consequently, iteration over a doubly circled set may not take place. The set or bag may be changed during the iteration, to allow for computations such as transitive closure. Since the system takes over after the construction of the loop body, which is also its initial interactive execution, ideally the user should be assured that the system-driven iteration will eventually terminate. Although termination of a Vizla iteration process is not guaranteed, this does not matter so much because the iteration set or bag may at all times be replaced by the object NULL, which causes early exit from the loop. On exit from the loop the iteration set or bag is restored to its original composition. This approach is preferable to the introduction of an "exit" operator because under our approach exit from the loop is uniformly from the first action of the loop body.

IV. THE COMPONENTS OF VIZLA

A query language causes operations to be performed on data that already exist in a data base. Vizla provides operations for numerical and boolean data types. For numerical data they are the four standard operations (PLUS, MINUS, MULT, and DIV), and for boolean data the operators are OR, AND, and NOT. Operator MINUS can be binary or unary (under the latter interpretation it changes the sign of a numerical entity). Moreover, for any data type based on a totally ordered set, one has comparison operators ($<$, $\leq$, $>$, $\geq$), and the MIN and MAX operations. For all data types there is equality ($=$) and nonequality ($\neq$).

Recall that Vizla is an interface to the prototyping language SF. An SF specification defines finite sets of abstract objects, and finite functions in terms of such objects. However, when an SF specification is converted into a prototype system, representations of the abstract objects have to be defined, and these are numbers, booleans, character strings, or tuples of the above. These data types are well understood.

For operations on sets Vizla provides union, intersection, and set subtraction. The subset predicate is also provided. There is no need for a separate membership predicate. Suppose we wanted to determine whether an element e belongs to a set S . Since the element can be interpreted as a singleton set when the context warrants it, $e \subseteq S$ is legitimate usage.

All the operations discussed above, with the exception of the NOT and the unary MINUS, are binary, and, if the binary operations are commutative and associative, they can be applied in a distributed sense to all elements of a set, e.g., addition applied to $\{6, 8, 9, 11\}$ gives 34. Such operations are addition and multiplication, the MIN and MAX, the AND and OR, and set union and intersection. Instead of creating a distributed operator for each of the above, we introduce the functional ALLOP that takes for its two operands an operator and a set. The operator is to be applied to the set distributively. Thus we have

$$\text{ALLOP}(\text{PLUS}; \{6, 8, 9, 11\}) = 34,$$

If set S of $\text{ALLOP}(op; S)$ is the null set, then ALLOP returns the unity element of op . A more detailed description of ALLOP can be found in [32]. In Vizla the set on which ALLOP is to

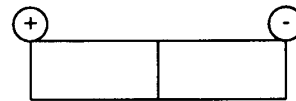


Fig. 3.

operate is always the workspace, and one merely points to ALLOP and to the appropriate operator. The result overwrites the workspace. Moreover, ALLOP may be used in Vizla with unary operators as well: unary MINUS and NOT is then applied to every element of the workspace.

Another general facility is NULL, which generates the null set. We have been pointing out that an object can function as an element and as a singleton set. The double interpretation applies also to null sets, but is somewhat more complicated. The null set is the unity element of set union. We shall therefore interpret it (in its guise of an "element") as the unity of the OR in boolean expressions (false), and of PLUS in arithmetic expressions (zero). In string comparisons it is to be interpreted as the null string.

A constant can be created by pointing to the operator CONST. The user is then prompted to type in the constant. This constant becomes the contents of the workspace.

Sometimes it is necessary to order data, and Vizla allows for this by providing sequences in addition to sets. The form of the sequence icon in a display of the operations is as shown in Fig. 3.

The two halves of the rectangle indicate the head (on the left) and tail (on the right) of the sequence. The circled symbols stand for push (+) and pop (-). Thus, to convert the sequence S from

$x678$

to

$678x$

we go through the operations

- circled minus → left rectangle → S
- circled plus → right rectangle → S .

The first three pointing actions change S from its original configuration to one in which its leftmost element has been removed. The operation is thus a mapping from sequences to sequences. However, there is the side effect that the removed element ends up in the workspace. The next series of actions pushes down the contents of the workspace at the right end of S . An empty sequence is created by NULL, but it cannot be identified as a sequence at that stage. This object becomes a sequence when the first push operation is performed on it. Pointing to a sequence without first defining a sequence operation brings the sequence into the workspace as an unordered bag.

A unary Vizla function SIZE returns the number of elements of a set or bag that is in the workspace. Thus, in terms of the authors and presenters example, the number of accepted papers is found by pointing to ACC of Fig. 2, which brings

this set into the workspace, and then to SIZE, which replaces the workspace with a count of the accepted papers. Since all SF functions are total, this count is also the cardinality of the function *Pres*. If SIZE is applied distributively, i.e., in conjunction with ALLOP, to a workspace consisting of individual elements, each element is replaced by 1; if it is applied to a collection of sets or bags, each set or bag is replaced by its size.

When we speak of the total number of authors of accepted papers, again there is more than one interpretation. Under one, if Jones is an author of three papers, Jones contributes three times to the total. Under another, Jones is counted just once. The following sequence of pointing actions gives the total under the first interpretation:

- 1) → ACC [WM displays ACC]
- 2) → Auth [WM displays Auth(ACC)]
- 3) → ALLOP
- 4) → SIZE [WM displays SIZE(Auth(ACC))]
- 5) → ALLOP
- 6) → PLUS [WM displays PLUS(SIZE(Auth(ACC)))]

After Actions 1 and 2 the workspace contains a collection of sets, one set of authors for each accepted paper. Now, if we were to apply SIZE directly to the workspace, the workspace would be replaced by a count of these sets, i.e., the number of accepted papers. We therefore modify SIZE with ALLOP, which causes each set in the workspace to be replaced by its cardinality. Application of ALLOP and PLUS adds the cardinalities, thus determining the total count of authors. This count under the second interpretation is given by

- 1) → ACC [WM displays ACC]
- 2) → Auth [WM displays Auth(ACC)]
- 3) → ALLOP
- 4) → UNION [WM displays UNION(Auth(ACC))]
- 5) → SIZE [WM displays SIZE(UNION(Auth(ACC)))]

The remaining Vizla operations relate to control. The operation START initiates a retrieval session, and END indicates its completion. We have already discussed SAVE and UPDATE, and ITERSTART and ITEREND. Operation UNSAVE can be used to remove sets from the temporary data base of the retrieval session. This is done by pointing to UNSAVE, and then to the circle that represents the set to be removed.

Suppose now that some action or sequence of actions is to take place only if a boolean condition is true. This is indicated by the definition of the boolean condition, followed by the sequence of actions, where the actions are bracketed by delimiters CONDSTART and CONDEND. An example of use will be given in the next section.

Let us consider the function *Auth* restricted to papers that have not been accepted. First we have to construct the restricted function. Pointing to *NAC* and then to *Auth* of Fig. 2 would produce a collection of the sets of authors of papers in *NAC*, but we want a set of ordered pairs of the form

< paper, set of its authors >

To get this set, use has to be made of the Vizla operator FUNCTION. The function is then generated in the workspace by the sequence of actions

- 1) → FUNCTION
- 2) → NAC
- 3) → Auth [WM displays Auth : NAC → SUB]

The → indicates that *Auth* is a set-valued function.

A filter may be applied to a set that is to be brought into the workspace. The filter is a boolean expression, and only those elements of the set for which the expression is true enter the workspace. Suppose we were to extract from ATT just those attendees whose names begin with B. This set is constructed as follows:

- 1) → FILTER [WM displays FILTER:]
- 2) → ≥ [WM displays FILTER: * ≥ _]
- 3) → CONST [User types "B"; WM displays FILTER: * ≥ "B"]
- 4) → AND [WM displays FILTER: (* ≥ "B") AND]
- 5) → < [WM displays FILTER: (* ≥ "B") AND (* < _)]
- 6) → CONST [User types "C"; WM displays FILTER: (* ≥ "B") AND (* < "C")]
- 7) → ATT [WM displays ATT | (* ≥ "B") AND (* < "C")]

Action 1 defines what follows as a filter, and Action 7 indicates the set to which the filter is to be applied. The filter is understood to have the form

$$(* \geq \text{"B"}) \wedge (* < \text{"C"})$$

An SF function has a single set for its domain, but the codomain may be a set, a cartesian product of sets, or a power set of the above. We now extend the filter concept to n-tuples. To take a simple example, let us find *Pres* restricted to presenters of papers whose names begin with A. One way of obtaining this set is as follows:

- 1) → FILTER [WM displays FILTER:]
- 2) → COORD [User types 2]
- 3) → < [WM displays FILTER: <1, * < _ >]
- 4) → CONST [User types "B"; WM displays FILTER: <1, * < "B" >]
- 5) → FUNCTION
- 6) → ACC
- 7) → Pres [WM displays Pres: ACC → ATT | <1, * < "B" >]

The filter is applied to function *Pres*, which is a set of ordered pairs of the form < paper, presenter >. Action 2 indicates that the filter is to be applied to coordinate 2 of the pair, and Action 3 that the constraint is to have the form of a "less than." The Workspace Monitor displays an ordered pair in which the 1 as its first coordinate is interpreted as saying that coordinate 1 is not affected by the filter, and the second coordinate is shown as * < _, where the underscore stands for a value that the user is to supply. This value is supplied by Action 4, and the filter is now defined: it is to let through only those pairs belonging to function *Pres* in which the second coordinate is less than "B," i.e., begins with "A." If a constraint were defined for coordinate 1 as well, then an ordered pair would pass through the filter only if both its coordinates were simultaneously to satisfy their respective constraints. This interpretation generalizes to n-tuples. For example, if the same

filter were applied to a quadruple, then the display after Action 4 would be FILTER: < 1, * < "B", 3, 4 >.

Next consider the finding of the inverse of a function. In SF, functions are always total, and the domain of a function is a set, but the codomain may be a set, a cartesian product of sets, or a power set of the above. Consider our function *Auth*. The inverse of this function as defined in mathematics would be a map from the collection of sets of authors that defines the range of *Auth* for a given domain (*PAP*, or some subset of *PAP*) to this particular domain. In SF and Vizla, on the other hand, the inverse of this function returns, for each author in the distributed union of the range of the function, the set of papers to which the author has contributed. Consider the function

$$\{ \langle P1, \{ \text{Jones, Brown} \} \rangle, \langle P2, \{ \text{Brown} \} \rangle, \langle P3, \{ \text{Jones} \} \rangle \}$$

In mathematics its inverse would consist of three ordered pairs which would differ from the function shown above merely by having their coordinates reversed. Moreover, if Brown had been the author of P3 as well, the function would have no inverse (a function cannot contain two pairs with the same first coordinate). The SF and Vizla inverse is

$$\{ \langle \text{Jones}, \{ P1, P3 \} \rangle, \langle \text{Brown}, \{ P1, P2 \} \rangle \}$$

Suppose the codomain of a function is a power set of triples. In this instance the inverse provides each triple in the distributed union of the range of the function with an image.

We also need a facility for restricting the papers returned by the inverse of *Auth* to just the accepted ones. The authors of accepted papers are found by the sequence of actions

$$\rightarrow ACC \rightarrow Auth.$$

The form of this sequence suggested to us that the required inverse should be constructed by the following sequence of actions:

$$\rightarrow INVERT \rightarrow ACC \rightarrow Auth \rightarrow SUB.$$

Operator INVERT always expects two sets to be indicated, one on each side of the function to be inverted. Here *ACC* and *SUB* are the two sets, and *Auth* is used to find the set of authors of accepted papers. Then, for each author of at least one accepted paper, all the accepted papers by this author are found.

Suppose that set *SUB* had been partitioned into subsets of submitters living in the United States (*US*) and not living in the United States (*NUS*). Now the sequence of actions

$$\rightarrow INVERT \rightarrow ACC \rightarrow Auth \rightarrow NUS$$

finds the set of authors of accepted papers as before, but this set is next intersected with *NUS*. Then all the accepted papers are found for each author belonging to the intersection. The order of the two sets is important, the domain of the function coming first. This is to avoid ambiguity when both indicated sets are subsets of the one set.

V. A MORE COMPLICATED QUERY

The fundamental objective for Vizla is the construction of a set in the workspace. For the examples of Section III this was straightforward—a function was applied to all elements of its domain or a restriction of the domain. At the end of Section IV we introduced the INVERT operator as a means for dealing with more complicated queries. We shall now deal with one such query: find presenters of papers who live in Canada and who present papers that have more than two authors or are more than ten pages long. Note again that we are not dealing with a normalized relational data base—our data are functions, and the functions may be set-valued.

Let us first find accepted papers that are more than ten pages long or have more than two authors. The sequence of actions

$$\rightarrow ACC \rightarrow Length$$

would return just a bag of integers, but, as discussed earlier, Vizla can be used to define inverses of functions. The actions

- 1) $\rightarrow$ START
- 2) $\rightarrow$ INVERT
- 3) $\rightarrow$ ACC
- 4) $\rightarrow$ Length
- 5) $\rightarrow$ INT [WM displays INV(Length(ACC)) WRT INT]

find the inverse of *Length*, i.e., for each length value the set of accepted papers of this length is returned. However, we are interested only in papers that have length greater than 10. This restriction can be imposed by means of a filter—we replace the last action shown above by

- 5) $\rightarrow$ FILTER [WM displays FILTER:]
- 6) $\rightarrow$ > [WM displays FILTER: * > _]
- 7) $\rightarrow$ CONST [User types 10; WM displays FILTER: * > 10]
- 8) $\rightarrow$ INT [WM displays INV(Length(ACC)) WRT INT | (* > 10)]

The collection of sets of papers now in the workspace is then collapsed into a single set, and the set saved as *P*:

- 9) $\rightarrow$ ALLOP
- 10) $\rightarrow$ UNION [WM displays UNION(previous display)]
- 11) $\rightarrow$ SAVE [User types *P*]

Next the papers with the appropriate number of authors is found. Because function *Auth* is set-valued, we have to iterate:

- 12) $\rightarrow$ NULL [WM displays NULL]
- 13) $\rightarrow$ SAVE
- 14) $\rightarrow$ ITERSTART
- 15) $\rightarrow$ ACC [WM displays $\in ACC$]
- 16) $\rightarrow$ SAVE
- 17) $\rightarrow$ Auth [WM displays $Auth(\in ACC)$]
- 18) $\rightarrow$ SIZE [WM displays $SIZE(Auth(\in ACC))$]
- 19) $\rightarrow$ > [WM displays $SIZE(Auth(\in ACC)) >]$
- 20) $\rightarrow$ CONST [User types 2; WM displays $SIZE(Auth(\in ACC)) > 2]$
- 21) $\rightarrow$ CONDSTART
- 22) $\rightarrow$ *16 [WM displays *16]
- 23) $\rightarrow$ UNION
- 24) $\rightarrow$ P [WM displays UNION(*16, *P*)]

- 25) → UPDATE
- 26) → *P*
- 27) → CONDEND
- 28) → ITEREND

Set *P* consisted of papers more than 10 pages long. Now to this set have been added papers with more than two authors (if they were not already in *P*). Finally the presenters of these papers (the papers are still in the workspace) are to be found, but only those presenters who live in Canada:

- 29) → *Pres* [WM displays *Pres*(UNION(*16, *P*))]
- 30) → SAVE
- 31) → INVERT
- 32) → *30
- 33) → *Country*
- 34) → CONST [User types "Canada"; WM displays *INV*(*Country*(*30)) WRT "Canada"]
- 35) → SAVE [User types *ANSWER*]
- 36) → END

VI. ADDITIONAL FEATURES

Perhaps the most important feature needed in a Vizla environment is a display of a trace of the pointing actions that have already been made, together with the corresponding workspace displays. A Vizla system can, of course, indicate syntactic errors, e.g., when one points to CONDSTART, but the workspace does not contain a boolean value. The removal of such a fault may require undoing several steps in the Vizla program, and a Vizla system should assist in this backtracking. Moreover, a failure of the program may have been caused by a fault far removed from the point at which failure is detected. Although the suitability of Vizla for programming in general is in doubt, so that a program of 73 actions, as in the Appendix, is unlikely to arise in practice, the retrieval sequence of Section V still comes to 36 actions, and failure detected at Action 35 may be due to a fault in Action 10, say. Note that the retrieval of papers longer than 10 pages ends with Action 11, and that there may be no need to undo any of the steps that follow Action 11. This means that there has to be an editing facility that can be applied to any part of a Vizla program.

Also, we have given no attention to the display of elements of sets or functions. Such a display facility would be essentially a test device. In testing the conceptual adequacy of a model or the appropriateness of a querying process one would commonly use a small test data base, and the sets generated by a querying process would be small enough to be displayed fully. Large sets can be given a partial representation. All SF data are ultimately numbers, text strings, or truth values, and these data are totally ordered. Suppose a set has 352 elements. A partial display of this set shows, say, the lowest ten elements under the total order, then the count 332 enclosed in a box, then the 10 highest elements. The display unit for a bag is a value and the number of occurrences of the value. For instance, the bag of truth values {T, F, T, T, F, F, T, F, T, T} has the representation [F, 4], [T, 7]. An alternative is to scroll the set of values past a window when the size of the set is too large for display of the entire set at once.

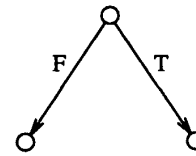


Fig. 4.

Vizla as a programming language could benefit by having a two-way branching facility. Although such a facility can be simulated by means of our CONDSTART/CONDEND feature, it may be convenient to introduce explicit branching. The branching facility requires operators BRANCH and CLOSE. A branch point is set up by defining a boolean condition and then pointing to BRANCH. This creates the display shown in Fig. 4.

Now, by pointing to either of the two lines originating from the root, one of the branches is selected for development. After the sequence of actions for this branch has been generated, the branch is closed off by pointing to CLOSE. The numbers of the actions bound to the branch are added to the display. Any branch that has not been closed off can be selected next for development. No example of branching is given here.

Another useful feature is a facility for reducing the arity of a set (or bag) of tuples in the workspace. This could be done by an operator REDUCE. Suppose the workspace consists of 5-tuples. The sequence of actions

- 1) → REDUCE
- 2) → COORD (user types 1)
- 3) → REDUCE
- 4) → COORD (user types 2)

replaces each quintuple $\langle x_1, x_2, x_3, x_4, x_5 \rangle$ with the triple $\langle x_2, x_4, x_5 \rangle$. Note that the second reduction is on the original coordinate 3 rather than 2 because the first reduction has already transformed the quintuples into quadruples.

In addition, we have not yet considered input and output, and the modularization of Vizla programs.

VII. DISCUSSION

The features introduced in Section VI have been given low priority because all our practical applications have been limited to conceptual design of information systems—the determination of whether the conceptual model of an application is adequate for the answering of all anticipated queries, the determination of which functions should be regarded as primary data, and the determination of what form these functions should take.

For example, in considering faculty and courses, our primary function could be the element-valued function *Instructor*, from courses to faculty, or the set-valued function *CoursesTaught*, from faculty to courses. With simplicity the guiding criterion we would give preference to an element-valued function. However, the defining of retrieval sequences may well show that the set-valued function is the more appropriate primary function.

Another example relates to ages of persons. Many examples in books and papers on conceptual modeling show *Age* as an attribute of *Person*. However, when we experiment with attributes that could be used to define each other, we find that *Age* can be defined in terms of the functions *DateOfBirth* and *DateNow*, where the latter returns the current date. Now, since the current date changes each day, *Age* also has to change. The appropriate primary datum is therefore the unchanging *DateOfBirth*. Moreover, this experimentation also shows that the subtraction in

$$Age = DateOfBirth - DateNow$$

has to be approached with care. Depending on its exact interpretation we could obtain *AgeLastBirthday*, *AgeNearestBirthday*, or *AgeNextBirthday*, all of which are used in insurance practice, but with obviously different effect on insurance premiums.

Next consider the statement "All Swedes receive support payments for their children." To understand this statement and its effect in an information system, one needs precise definitions of "Swede" and "child" for this particular context. For argument's sake, in this context, "Swede" could refer to a resident of Sweden of at least six months' standing. We might find, however, that our conceptual model contains no data relating to a person's period of residence.

These examples show that Vizla can be used very effectively in a paper-and-pencil mode to check the completeness and convenience of the conceptual model of an application, such as that of Fig. 2, but even at this level of usage Vizla is still experimental. Nevertheless, a few general observations can be made. Most importantly, as we already pointed out before, we see its primary use in retrieval of information from functional data bases during the prototyping of information systems. Hence the shortest path algorithm of the Appendix is not a typical application of Vizla. But it does show that Vizla has sufficient expressive power to deal with fairly complicated problems, and closer examination of the program shows that Vizla should not be dismissed out of hand as a general programming language. Although the listing of the 73 actions is rather long, the program is generated with just 43 pointing actions to data representations, 24 keystrokes for defining saved data objects, another 4 keystrokes for indicating that in four cases the system is to assign identifiers to saved data, and 30 pointing actions to Vizla operators (or 30 keystrokes if we had a special keyboard).

Moreover, the general sequence operations could be replaced by specialized stack or queue operations, where each stack or queue would be represented by a two-component symbol, and pointing to one or the other of the components would result in a push from or a pop to the workspace. This modification would reduce the actions in the program of the Appendix to 65. Of course, one has to be careful to balance economy of concepts against the wish to have shorter programs. Therefore, any streamlining, such as the replacement of the general sequence with specializations, has to be postponed until practical experience has been gained.

The only Vizla operations for which we have designed an iconic representation are the sequence operations, and we have

taken over standard mathematical symbols for the comparison operators. For the other operations individual users can design iconic representations with which they feel most comfortable. For example, as regards set intersection, some users may prefer the symbol $\cap$, others the textual icon INTER, still others might prefer a Venn diagram of two intersecting sets in which the intersection is shaded.

What does Vizla achieve? The ISA relation and subset partitions in a schema diagram alert a user to the potential of domain-conditioned ambiguity. Since such ambiguity is entirely dependent on how a user interprets the terminology of a particular domain, this is as far as we can go. Also, a Vizla schema diagram merely shows functions, and leaves their implementation open. For example, the sunrise function for Pittsburgh for the year 1933, which is a function from 365 dates to times of day, could be implemented as a computational rule (a function subprogram) or as a table. A Vizla user does not have to know which implementation has been adopted. As regards user convenience, the Vizla workspace concept allows a user to separate querying processes into sequences of small steps, save intermediate results of such sequences, and combine the intermediate results into final results. Such a divide-and-conquer approach is necessary because Vizla allows ad hoc queries of arbitrary complexity, leading to querying processes as complicated as the function subprogram for the Pittsburgh sunrises. This differentiates Vizla from conventional query languages. The latter, which are ultimately based on first-order predicate calculus, cannot deal with transitive closure of a relation [33]. This applies also to the visual query languages of Larson [22] and the Tampere group [23]. Vizla iteration solves the transitive closure problem.

The first priority in our future work is to be given to a study of the optimization of queries and Vizla programs. Straightforward translation of the pointing actions into code can result in levels of efficiency so low as to be acceptable only for prototypes. Although it would be feasible to develop efficient Vizla programs, they might have to be so complicated that the edge Vizla programming has over textual programming as regards convenience would be lost. Consequently we shall look to mechanized performance-enhancing transformations as a means of improving efficiency. This work could be based on a study of the representations generated by the Workspace Monitor.

On the negative side, it must be noted that the construction of a query sequence by pointing actions need not appeal to all classes of users. Also, the 36 actions for one of our queries seem an excessive number. One possibility for cutting down on the number of operations would be to present the system with a pseudo-query. Some work on pseudo-queries has been done by Motro [34], who refers to the components of a pseudo-query as tokens. For the examples of Section III the tokens of the pseudo-query would be the set of presenters and the complement of the set of authors. The tokens could serve as a base from which the system would construct candidate queries for the user's inspection. The user would then select the proper query from the set of alternatives prepared by the system.

It should be possible to translate queries freely from one representational form to another. When a querying process

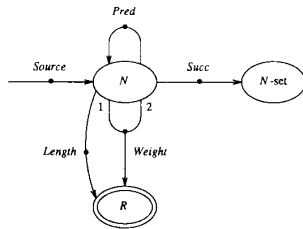


Fig. 5. Type structure for the shortest paths problem.

is defined in terms of pointing actions, the representation generated by the Workspace Monitor could be used to construct a corresponding logical expression as well. The logical expression should in turn be expressible as a natural language paraphrase. Indeed, experience with the paraphrasing could lead to the definition of a restricted subset of natural language which would give more structure to the pseudo-queries discussed in the preceding paragraph.

Here we have been looking at queries formulated during the prototyping stage in the development of an information system. We have therefore required strict discipline in the formulation of queries. During the operation of an information system the perspective changes somewhat. The concern is no longer a test of completeness of the data base, but the extraction of information needed to run an enterprise. Motro has investigated ways of making an information system respond to a user's needs when the user has no proper understanding of the data base [35], or even of his or her own information needs [36]. A query facility based on a visualized conceptual model of an information system seems very appropriate for dealing with these issues.

APPENDIX

VIZLA AS A PROGRAMMING LANGUAGE

The sequences of pointing actions in Vizla do not actually define queries. Instead, sets are constructed that contain the data the user wants to see. If the sequences of pointing actions were converted into programs, these programs could be optimized and saved for reuse. The preservation of pointing actions as programs is important for Vizla as a data base retrieval language, but it also makes Vizla a general programming language.

Our example is Dijkstra's algorithm for shortest paths from a single source [37]. We emphasize that Vizla is not intended for programming in general, particularly not for anything as complicated as Dijkstra's algorithm. However, this application serves as a useful demonstration: if Vizla can cope with Dijkstra's algorithm, then it should be able to cope with queries of arbitrary complexity. The data base for this application consists of the sets and functions shown in Fig. 5, where we have used a representational style popular for abstract data types (see, e.g., [38]).

In Fig. 5, N is the set of nodes of the digraph, and $N\text{-set}$ is its power set. For each node n , the function $Succ$ identifies the set of nodes at which arcs originating at n terminate, and the function $Weight$, which is defined only for arcs of the digraph,

gives the length associated with the arc as a real number. The source node is identified by the constant function $Source$.

The mutable function $Length$, from nodes to reals, ultimately holds the lengths of the shortest paths, and function $Pred$ then allows the paths to be explicitly defined by the reverse of the sequence $Pred(n)$, $Pred(Pred(n))$, The visual program for Dijkstra's algorithm now follows.

- 1) $\rightarrow$ START
- 2) $\rightarrow N$
- 3) $\rightarrow$ SAVE [As Q]
- 4) $\rightarrow$ ITERSTART
- 5) $\rightarrow N$

The initial actions create Q as a copy of set N , and initiate iteration over N . In Actions 6–17 a search is made for the element in Q with the smallest value of $Length$. This element is saved as $NODE$, and nodes directly reachable from $NODE$ are saved as $ENDS$ (Actions 18–20). Element $NODE$ is then removed from Q (Actions 21–25).

- 6) $\rightarrow Q$
- 7) $\rightarrow Length$
- 8) $\rightarrow$ ALLOP
- 9) $\rightarrow$ MIN
- 10) $\rightarrow$ SAVE
- 11) $\rightarrow$ INVERT
- 12) $\rightarrow Q$
- 13) $\rightarrow Length$
- 14) $\rightarrow$ FILTER
- 15) $\rightarrow$ =
- 16) $\rightarrow *10$
- 17) $\rightarrow R$
- 18) $\rightarrow$ SAVE [As $NODE$]
- 19) $\rightarrow Succ$
- 20) $\rightarrow$ SAVE [As $ENDS$]
- 21) $\rightarrow Q$
- 22) $\rightarrow$ DIFF
- 23) $\rightarrow NODE$
- 24) $\rightarrow$ UPDATE
- 25) $\rightarrow Q$.

Next, in an iteration over $ENDS$, the weights on all arcs originating from $NODE$ are extracted (Actions 31–35), and for each arc $\langle NODE, n_i \rangle$ we determine in Actions 37–41:

$$Length(NODE) + Weight(NODE, n_i).$$

If $ENDS$ is empty, the iteration sequence defined by Actions 26–71 is skipped.

- 26) $\rightarrow$ ITERSTART
- 27) $\rightarrow ENDS$
- 28) $\rightarrow$ SAVE [As $ENDNODE$]
- 29) $\rightarrow Length$
- 30) $\rightarrow$ SAVE
- 31) $\rightarrow 1$ [This is the line labeled 1 in Fig. 3]
- 32) $\rightarrow NODE$
- 33) $\rightarrow 2$ [This is the line labeled 2 in Fig. 3]
- 34) $\rightarrow ENDNODE$
- 35) $\rightarrow Weight$ [WM displays $Weight(NODE, ENDNODE)$]
- 36) $\rightarrow$ SAVE

37) $\rightarrow NODE$
 38) $\rightarrow Length$
 39) $\rightarrow PLUS$
 40) $\rightarrow *36$
 41) $\rightarrow SAVE$

Next we replace $Length(n_i)$ by $Length(NODE) + Weight(NODE, n_i)$ if the latter is the smaller of the two, and make an appropriate change to $Pred(n_i)$ as well. In previous examples we used UPDATE to replace one set by another. We shall now need to replace functions by their modified versions, but first we have to generate the new elements that will take part in the modifications. Sequence operations will be used to generate these elements (which are ordered pairs here).

42) $\rightarrow <$
 43) $\rightarrow *30$
 44) $\rightarrow CONDSTART$
 45) $\rightarrow NULL$
 46) $\rightarrow SAVE [as L]$
 47) $\rightarrow NULL$
 48) $\rightarrow SAVE [as P]$
 49) $\rightarrow ENDNODE$
 50) $\rightarrow$ circled plus
 51) $\rightarrow$ right rectangle
 52) $\rightarrow L$
 53) $\rightarrow$ circled plus
 54) $\rightarrow$ right rectangle
 55) $\rightarrow P$
 56) $\rightarrow *41$
 57) $\rightarrow$ circled plus
 58) $\rightarrow$ right rectangle
 59) $\rightarrow L$
 60) $\rightarrow NODE$
 61) $\rightarrow$ circled plus
 62) $\rightarrow$ right rectangle
 63) $\rightarrow P$.

Now L and P hold ordered pairs that are to replace existing ordered pairs in functions $Length$ and $Pred$, respectively.

We introduce the map overwrite operation MAPOVER (not shown in Fig. 2). In terms of function F with functionality

$$F : S_1 \times S_2 \times \dots \times S_k \rightarrow S_{k+1} \times \dots \times S_n$$

suppose MAPOVER is provided with the argument $< a_1, a_2, \dots, a_k, a_{k+1}, \dots, a_n >$. If no n -tuple of the form $< a_1, a_2, \dots, a_k, \dots >$ belongs to F , then the argument of MAPOVER is added to F ; otherwise the old $< a_1, a_2, \dots, a_k, \dots >$ is replaced with the argument of MAPOVER. The usage is as follows: the n -tuple that is the argument of MAPOVER is expected to be in the workspace; the user points to MAPOVER and to the representation of the function that is to be changed.

The next sequence of actions finishes our example:

64) $\rightarrow L$
 65) $\rightarrow$ MAPOVER
 66) $\rightarrow Length$
 67) $\rightarrow P$
 68) $\rightarrow$ MAPOVER
 69) $\rightarrow Pred$

70) $\rightarrow$ CONDEND
 71) $\rightarrow$ ITEREND
 72) $\rightarrow$ ITEREND
 73) $\rightarrow$ END.

ACKNOWLEDGMENT

The author thanks Janis Bubenko, Shi-Kuo Chang, and Hannu Kangassalo for introducing him to conceptual models of information systems, visual languages, and visual queries, respectively. A disagreement with John Sowa on the merits of natural language for query formulation convinced him of the need for the work reported here.

REFERENCES

- [1] D. Rombach, "Software specification: A framework," *Curriculum Module SEI-CM-11-1.0*. Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 1987.
- [2] M.A. Jackson, *System Development*. Englewood Cliffs, NJ: Prentice-Hall, 1983.
- [3] E. Dubois, J. Hagelstein, E. Lahou, F. Ponsaert, A. Rifaut, and F. Williams, "The ERAE model: A case study," in *Information Systems Design Methodologies: Improving the Practice*, T.W. Olle, H.G. Sol, and A.A. Verrijn-Stuart, Eds. Amsterdam: North-Holland, 1986, pp. 87-105.
- [4] J. Fiadeiro and A. Sernadas, "The INFOLOG linear tense propositional logic of events and transactions," *Inf. Systems*, vol. 11, pp. 61-85, 1986.
- [5] A.I. Wasserman, P.A. Pircher, D.T. Shewmake, and M.L. Kersten, "Developing interactive information systems with the User Software Engineering methodology," *IEEE Trans. Software Eng.*, vol. SE-12, pp. 326-345, 1986.
- [6] A.T. Berztiss, "Data abstraction in the specification of information systems," in *Proc. IFIP World Congress 86*, pp. 83-90, 1986.
- [7] T.W. Olle, J. Hagelstein, I.G. Macdonald, C. Rolland, H.G. Sol, F.J.M. Van Assche, and A.A. Verrijn-Stuart, *Information System Methodologies—A Framework for Understanding*. Wokingham, England: Addison-Wesley, 1988.
- [8] M. Jarke and Y. Vassiliou, "A framework for choosing a database query language," *ACM Comp. Surveys*, vol. 17, pp. 313-340, 1985.
- [9] J. Clement, J. Lochhead, and E. Soloway, "Positive effects of computer programming on students' understanding of variables and equations," in *Proc. ACM Annual Conf. 1980*, pp. 467-474.
- [10] C. Welty and D.W. Stemple, "Human factors comparison of a procedural and a nonprocedural query language," *ACM Trans. Database Syst.*, vol. 6, pp. 626-649, 1981.
- [11] G.W. Hansen and J.V. Hansen, "Procedural and nonprocedural query languages revisited: a comparison of relational algebra and relational calculus," *Int. J. Man-Machine Studies*, vol. 26, pp. 683-694, 1987.
- [12] W. Swartout, "GIST English generator," in *Proc. AAAI Natl. Conf. 1982*, pp. 404-409.
- [13] W.S. Luk and S. Kloster, "ELFS: English language from SQL," *ACM Trans. Database Syst.*, vol. 11, pp. 447-472, 1986.
- [14] B. Czejdo, R. Elmasri, M. Rusinkiewicz, and D.W. Embley, "Graphical query languages for semantic database models," in *Proc. 1987 Natl. Computer Conf.*, pp. 615-623.
- [15] B. Czejdo, D. Reddy, and M. Rusinkiewicz, "Design and implementation of an interactive graphical query interface for a relational database management system," in *Proc. 1988 IEEE Workshop on Visual Languages*, pp. 14-20.
- [16] D. Harel, H. Lachover, A. Naamad, A. Pnueli, M. Politi, R. Sherman, and A. Shtul-Trauring, "Statemate: a working environment for the development of complex reactive systems," in *Proc. 10th Int. Conf. Software Eng.*, pp. 396-406, 1988.
- [17] H.J. Kim, H.F. Korth, and A. Silberschatz, "Picasso: a graphical query language," *Software Pract. Exp.*, vol. 18, pp. 169-203, 1988.
- [18] H.K.T. Wong and I. Kuo, "GUIDE: Graphical user interface for database exploration," in *Proc. VLDB 1982*, pp. 22-32.
- [19] Z.Q. Zhang, M.O. Alberto, and O. Mendelzon, "A graphical query language for entity-relationship databases," in *Entity-Relationship Approach to Software Engineering*, C.G. Davis, S. Jajodia, P.A.B. Ng, and R.T. Yeh, Eds. Amsterdam: North-Holland, 1983, pp. 441-448.
- [20] M. Angelaccio, T. Catarci, and G. Santucci, "QBD: A fully visual system for E-R oriented databases," in *Proc. 1989 IEEE Workshop on Visual Languages*, pp. 56-61.

- [21] L.E. Echeverria and J.A. Pino, "An intuitive approach for the expression of Boolean queries," in *Proc. 1989 IEEE Workshop on Visual Languages*, pp. 118-123.
- [22] J.A. Larson, "Visual languages for database users," in *Visual Languages*, S.K. Chang, T. Ichikawa, and P.A. Ligomenides, Eds. New York: Plenum Press, 1986, pp. 127-147.
- [23] S. Kari, H. Kangassalo, and J. Poso, "Conceptual query language CQL: a visual user interface to application data bases," to appear in *Proc. Joint Scandinavian-Japanese Seminar on Information Modeling and Knowledge Bases*.
- [24] H. Kangassalo, "Concept D: A graphical language for conceptual modeling and data base use," in *Proc 1988 IEEE Workshop on Visual Languages*, pp. 2-11.
- [25] R.B. Grafton and T. Ichikawa, Eds., special issue on visual programming, *Computer*, vol. 18, no. 6, Aug. 1985.
- [26] D.L. Parnas, "On the criteria to be used in decomposing systems into modules," *Comm. ACM*, vol. 15, pp. 1053-1058, 1972.
- [27] J.R. Cameron, "An overview of JSD," *IEEE Trans. Software Eng.*, vol. SE-12, pp. 222-240, 1986.
- [28] M.W. Alford, "SREM at the age of eight; the distributed computing design system," *Computer*, vol. 18, no. 4, pp. 36-46, Apr. 1985.
- [29] M. Graf, "Building a visual designer's environment," in *Principles of Visual Programming Systems*, S.-K. Chang, Ed. Englewood Cliffs, NJ: Prentice-Hall, 1989, pp. 291-325.
- [30] D.W. Shipman, "The functional data model and the data language DAPLEX," *ACM Trans. Database Syst.*, vol. 6, pp. 140-173, 1981.
- [31] A.T. Berztiss, "Formal specification methods and visualization," in *Principles of Visual Programming Systems*, S.-K. Chang, Ed. Englewood Cliffs, NJ: Prentice-Hall, 1989, pp. 231-290.
- [32] A.T. Berztiss, "A notation for distributed operations," *Inf. Proc. Letters*, vol. 26, pp. 19-21, 1987.
- [33] A.V. Aho and J.D. Ullman, "Universality of data retrieval languages," in *Proc. 6th ACM Symp. on Principles of Prog. Languages*, pp. 110-117.
- [34] A. Motro, "Constructing queries from tokens," in *Proc. ACM SIGMOD 1986 Int. Conf. Management Data*, pp. 120-131.
- [35] ———, "SEAVE: A mechanism for verifying user presuppositions in query systems," *ACM Trans. Office Inf. Syst.*, vol. 4, pp. 312-330, 1986.
- [36] ———, "VAGUE: a user interface to relational databases that permits vague queries," *ACM Trans. Office Inf. Syst.*, vol. 6, pp. 187-214, 1988.
- [37] E.W. Dijkstra, "A note on two problems in connection with graphs," *Numer. Math.*, vol. 1, pp. 269-271, 1959.
- [38] J.A. Goguen, J.W. Thatcher, and E.G. Wagner, "An initial algebra approach to the specification, correctness, and implementation of abstract data types," in *Current Trends in Programming Methodology*, Vol. IV: Data Structuring, R.T. Yeh, Ed. Englewood Cliffs, NJ: Prentice-Hall, 1978, pp. 80-149.



Alfs Berztiss received the Ph.D. degree in physics in 1964 from the University of Melbourne, Australia.

He is on the faculty of the Department of Computer Science at the University of Pittsburgh, and is a research Associate of SYSLAB, University of Stockholm. His research interests include prototyping of information and control systems, uncertainties in decision support systems, and curriculum development for software engineering. He has developed curriculum material for SEI and has presented numerous courses on various aspects of software engineering to universities and industry in many countries. He is the author of *Data Structures: Theory and Practice* (Academic, 1971 and 1975), *Programming with Generators* (Ellis Horwood, 1990), and numerous technical papers.

Dr. Berztiss is a Fellow of the Australian Computer Society, and a member of IFIP Working Group 8.1 (Design and Evaluation of Information Systems), ACM, IEEE CS, and EATCS.